ANALYSIS OF AI/ML MODEL FAILURES AND PERFORMANCE IMPROVEMENT USING RECOVERY TECHNIQUES

A PROJECT REPORT SUBMITTED TO BHARATHIAR UNIVERSITY IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF

BACHELOR OF DATASCIENCE

Submitted By

P. SUYA BHARATHI

(**Reg. No. 2328M0118**)

Under the Guidance

Dr.R.KARTHIK MCA., Ph.D

SRI KRISHNA ADITHYA COLLEGE OF ARTS & SCIENCE

Recognized by UGC with 2(f) & 12(B) status | Affiliated to Bharathiar University

Kovaipudur, Coimbatore - 641042

SRI KRISHNA INSTITUTIONS

March 2026

CERTIFICATION

This is to certify that the dissertation , entitled "**Analysis of AI/ML Model Failures and Performance Improvement Using Recovery Techniques"**, submitted to the **BHARATHIAR UNIVERSITY** , in partial fulfillment of the requirement for the award of the **Degree of BACHELOR OF DATA SCIENCE** , Is a record of original research work done by Ms**. P .SUYABHARATHI** during the period 2025-2026 of her study in the Department of Data science, Sri Krishna Adithya College of Arts and Science, Coimbatore under my supervision and guidance and the project work has not formed the basis for the award of any Degree / Diploma / Associate ship / Fellowship or other similar title to any candidate of any University.

SIGNATURE OF THE GUIDE **SIGNATURE OF THE HOD**

SIGNATURE OF THE PRINCIPAL

Viva-Voice conducted on :_____

EXTERNAL EXAMINER INTERNAL EXAMINER

DECLARATION

I, **P.SUYABHARATHI** hereby declare that the project work, entitled **"ANALYSIS OF AI/ML MODEL FAILURES AND PERFORMANCE IMPROVEMENT USING RECOVERY TECHNIQUES"** submitted to the Sri Krishna Adithya College of Arts and Science, in partial fulfilment of the requirements for the award of the Degree of **BACHELOR OF DATA SCIENCE**, is a record of original research work done by me during the period 2025-2026 under the supervision and guidance of **Dr.R.KARTHIK MCA.,Ph.D ASSISTANTPROFESSOR, DEPARTMENT OF DATA SCIENCE** of, Sri Krishna Adithya College of Arts and Science, Coimbatore and it has not formed the basis for the award of any Degree / Diploma / Associate ship / Fellowship or other similar title to any candidate of any University. Place: Coimbatore Signature of the Candidate

Date:

(P.SUYABHARATHI)

ACKNOWLEDGEMENT

First and foremost, I would like to express my heartfelt gratitude to Almighty God for His endless blessings, guidance, and strength throughout the course of this project. His grace has enabled me to persevere and successfully complete this work.

The successful completion of my project would not have been possible without my parent's support. I take this opportunity to thank for their continuous encouragement.

I take this opportunity to express my sincere and whole hearted thanks to **Dr.S. PALANIAMMAL, MSc, M.Phil., Ph.D.,** Principal, Sri Krishna Adithya College of Arts and Science, Coimbatore for reinforcing my effort with support and guidance.

I owe my heart and profound gratitude to **Dr.K. GEETHA, MSc, M.Phil., Ph.D., DEAN OF COMPUTER SCIENCE**, Sri Krishna Adithya College of Arts and Science, Coimbatore for her continuous support. My internal guide **Dr.R.Karthik,MCA,Ph.D,Assistant Professor, DEPARTMENT OF DATA SCIENCE,** Sri Krishna Adithya College of Arts and Science, Coimbatore for giving continuous encouragement to complete this work. I appreciate the assistance to **Dr.P.Jayasheelan MCA., PHD.,Assistant Professor** for their encouragement and insights throughout the entire process. Their constructive feedback and constant support have been pivotal in shaping this project.

I am grateful to thank all the faculties in our department for their motivation and endless inspiration to help in completing this project work. I convey my thankfulness to all my friends who were with me to share my happiness and agony

P. SUYA BHARATH

CONTENTS

CHAPTER NO	TITLE	PAGE
1.	**INTRODUCTION**	6-10
	1.1 Synopsis	6
	1.2 About the project	7
	1.3 Introduction	8
	1.4 Data Overview	9
2.	**DATA PREPROCESSING**	11-16
	2.1 Data cleaning	12
	2.2 Text Data Pre-processing	14
	2.3 Data Aggregation	15
	2.4 Normalization and scaling	15
	2.5 Data Splittling	16
3.	**DATA VISUALIZATION**	17-26
	3.1 Visualization	17
	3.2 Correlation	25
4.	**CONCLUSION**	26-28
	4.1 Conclusion	26
	4.2 Future Enhancement	27
	4.3 References	28

1.1 SYNOPSIS

The goal of this project, Analysis of AI/ML Model Failures and Performance Improvement Using Recovery Techniques, is to examine the typical causes of AI/ML model failures and how appropriate recovery techniques can enhance their performance. Numerous fields, including computer vision, natural language processing, healthcare, and finance, make extensive use of AI and ML models. However, concerns like poor data quality, class imbalance, overfitting, noisy labeling, and unstable training frequently cause these models to perform poorly. To increase the dependability and efficiency of AI systems, it is critical to comprehend these problems.

This project analyzes model failures in many areas using the Synthetic AI Model Failure Recovery Dataset. Model kinds, domain categories, failure reasons, and performance evaluation measures are all included in the dataset. The project uses this information to determine which machine learning models fail more frequently, the domains in which failures are more common, and the primary causes of performance decline.

Additionally, the study investigates several recovery strategies that might be used to enhance model performance following failure. Techniques like threshold optimization, architectural modification, class rebalancing, and data augmentation are studied. To gauge the improvement, the models' performance is assessed both before and after using these recovery strategies.Pandas, NumPy, Matplotlib, Scikit-learn, and other Python libraries are used to implement the analysis in Google Colab. The study's findings offer insightful information on AI model failures and show how suitable recovery techniques can enhance machine learning systems' dependability and performance.

1.2 ABOUT THE PROJECT

The purpose of this project, Analysis of AI/ML Model Failures and Performance Improvement Using Recovery Techniques, is to investigate the reasons behind the failure of AI and ML models as well as ways to enhance their performance. Appropriate training and high-quality data are essential for machine learning models. The model may produce inaccurate predictions or subpar outcomes if the data is unbalanced or contains flaws. Common issues such data imbalance, missing values, noisy labels, and overfitting are examined in this paper. AI/ML models may perform worse as a result of these issues. The Synthetic AI Model Failure Recovery Dataset, which includes data on various model types, domains, training problems, and performance outcomes, is used in the research.

The project aims to analyze the behavior of different AI/ML models and understand why they fail under certain conditions. Machine learning models depend heavily on data quality and proper training procedures. When these factors are not optimal, models may produce inaccurate predictions or unstable results.This study investigates various failure scenarios including data imbalance, missing values, noisy labels, and overfitting. The dataset used for this project contains multiple attributes describing the model type, domain, training issues, and performance metrics.

In order to enhance model performance, the research also investigates various recovery strategies. To address model failures, techniques like oversampling, SMOTE rebalancing, data augmentation, and model architectural modifications are employed.Metrics including accuracy, precision, recall, F1-score, and ROC-AUC are used to assess the models' performance. The project demonstrates how model performance can be enhanced by comparing the outcomes before and after recovery procedures are applied.

1.3 INTRODUCTION

In several modern businesses, artificial intelligence has grown in importance as a technology. Numerous industries, including healthcare, banking, image identification, and text analysis, make extensive use of machine learning models. These models are useful for tasks including sentiment analysis, picture categorization, fraud detection, and disease prediction. Despite their great use, machine learning models can occasionally fail for a variety of reasons.

Poor data quality, class imbalance, insufficient training data, and model overfitting are some frequent causes of model failure. These issues may make AI systems less accurate and dependable. As a result, it's critical to investigate the reasons behind models' failures and devise strategies for enhancing their functionality.

Several machine learning models are examined in this study, Analysis of AI/ML Model Failures and Performance Improvement Using Recovery Techniques, to determine how different factors impact their performance. Additionally, the study investigates recovery strategies that can enhance model performance following failure.Python is used in the Google Colab environment to implement the project. Libraries like Pandas, NumPy, and Matplotlib are used for data processing and visualization. Graphs and charts are used to effectively illustrate the model failure patterns and performance enhancements.

1.4 DATA OVERVIEW

The dataset used in this project contains approximately **6,000 records with 24 variables** related to Artificial Intelligence and Machine Learning model experiments across different domains. It includes information about **model types, data quality conditions, training behavior, and performance metrics**.Key variables such as **class imbalance ratio, label noise level, missing data ratio, and overfitting score** help identify the main causes of model failure. Performance metrics like **accuracy, precision, recall, F-score, and ROC-AUC** are used to evaluate model performance.The dataset also includes **failure types, recovery techniques, and post-fix F score**, which help analyze how recovery methods improve AI model performance.

COLUMN DESCRIPTION:

Column Name	Description
Project_id	Unique identifier for each AI model experiment
Year	Year in which the experiment or model evaluation was conducted
Domain	Application domain where the model is used (e.g., healthcare, finance, computer vision)
Model_type	Type of machine learning or deep learning model used
Class_imbalance_ratio	Percentage of incorrect or noisy labels in the dataset
Missing_data_ratio	Proportion of missing values present in the dataset
Data_leakage	Indicates whether data leakage occurred during model training
Overfitting_score	Measurement of how much the model overfits the training data

Unstable_training	Indicates instability during the training process
Train_val_gap	Difference between training and validation performance
Unstable_training	Indicates instability during the training process
Underfitting_flag	Shows whether the model suffers from underfitting
Accuracy	Overall prediction accuracy of the model
Precision	Precision metric measuring correct positive predictions
Recall	Recall metric measuring the ability to identify positive cases
F_score	Harmonic mean of precision and recall
Roc_auc	Area under the ROC curve measuring classification performance
Failure_type	Main reason for the model failure
Applied_augmentation	Indicates whether data augmentation was applied
Rebalancing_method	Technique used to balance the dataset (e.g., SMOTE, oversampling)
Architecture_change	Indicates if the model architecture was modified
Threshold_optimization	Indicates whether prediction threshold optimization was applied
Post_fix_f_score	F score obtained after applying recovery techniques
Improvement_flag	Indicates whether the recovery technique improved performance

1.4 DATA PREPROCESSING

To preprocess the dataset, missing values are identified and handled using appropriate methods such as imputation or removal. Categorical variables like **domain, model type, failure type, and recovery technique** are converted into numerical form using encoding techniques. Duplicate records are checked and removed to maintain data consistency. Continuous variables such as **class imbalance ratio, label noise level, missing data ratio, and performance metrics** are scaled or normalized to ensure uniform data ranges.Unnecessary or redundant columns are removed, and optional steps like **feature engineering and outlier detection** may be applied to prepare the dataset for analysis.

1.Import Libraries and Load the Dataset:

The first step of the analysis process is to import the required Python libraries and load the dataset. The dataset is read using the pandas library and stored in a DataFrame. After loading the dataset, the first few rows and the overall structure of the data can be viewed. This helps to understand the dataset, including the column names, data types, and any missing values before performing further analysis.

Code:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

sns.set(style="whitegrid")

df = pd.read_csv("/content/synthetic_ai_model_failure_recovery.csv")

df.head()
```

Output:

	project_id	year	domain	model_type	dataset_size	class_imbalance_ratio	label_noise_level	missing_data_ratio
0	PRJ_1	2025	finance	XGBoost	41590	0.353677	0.046806	0.038999
1	PRJ_2	2020	computer_vision	CNN	69592	0.110945	0.136821	0.196294
2	PRJ_3	2023	finance	CNN	10192	0.376237	0.077634	0.165631
3	PRJ_4	2024	finance	DenseNet	3068	0.107264	0.084280	0.135674
4	PRJ_5	2018	medical	Transformer	98518	0.230751	0.093295	0.081296

data_leakage	overfitting_score	...	precision	recall	f1_score	roc_auc	failure_type	applied_augmentation	rebalancing_method
0	0.969910	...	0.591702	0.652121	0.343307	0.783665	class_imbalance	GAN	none
0	0.170524	...	0.904199	0.652307	0.452734	0.634185	model_complexity	none	oversampling
1	0.775133	...	0.960937	0.544246	0.496674	0.668594	model_complexity	SMOTE	none
0	0.198716	...	0.864504	0.885635	0.386040	0.625916	model_complexity	GAN	none
0	0.713245	...	0.746898	0.761366	0.646209	0.749639	label_noise	GAN	undersampling

failure_type	applied_augmentation	rebalancing_method	architecture_change	threshold_optimization	post_fix_f1_score
class_imbalance	GAN	none	0	0	0.513530
model_complexity	none	oversampling	0	0	0.621217
model_complexity	SMOTE	none	0	0	0.656016
model_complexity	GAN	none	0	0	0.450950
label_noise	GAN	undersampling	0	0	0.873651

2. DATA CLEANING:

Data cleaning, which prepares the dataset for analysis by fixing problems like missing values, duplicate entries, and inconsistent data types, is one of the most crucial stages in data analysis. If these issues are not appropriately addressed, they may have an impact on the analysis's accuracy. The data for this project dataset was examined to find any missing or inaccurate values, and the necessary actions were made to address them. Additionally, duplicate records were reviewed and eliminated if needed. To guarantee uniformity throughout the dataset, each column's data types were also confirmed. The dataset's quality and dependability are enhanced by these data cleaning procedures, making it appropriate for additional analysis of AI model failures and performance enhancement.

1.Checking for Missing Values

The dataset's columns are checked for missing values using the method df.isnull().sum(). It counts how many null or missing values there are in each column. This makes it easier to determine which columns have missing data so that the necessary actions can be made to address them prior to additional analysis.

Code:

```
df.isnull().sum()
```

Output:

project_id	0	data_leakage	0	roc_auc	0
year	0	overfitting_score	0	failure_type	0
domain	0	train_val_gap	0	applied_augmentation	0
model_type	0	unstable_training	0	rebalancing_method	0
dataset_size	0	underfitting_flag	0	architecture_change	0
class_imbalance_ratio	0	accuracy	0	threshold_optimization	0
label_noise_level	0	precision	0	post_fix_f1_score	0
missing_data_ratio	0	recall	0		

2.Handle Missing Values

The dataset's missing values are handled by this code. It starts by choosing every numerical column in the dataset. The mean value of the corresponding column is then used to replace any missing values in those columns. This keeps the dataset intact and guarantees that it is prepared for additional analysis.

Code:

```
num_cols = df.select_dtypes(include=np.number).columns
df[num_cols] = df[num_cols].fillna(df[num_cols].mean())
df.columns
```

Output:

```
Index(['project_id', 'year', 'domain', 'model_type', 'dataset_size',
       'class_imbalance_ratio', 'label_noise_level', 'missing_data_ratio',
       'data_leakage', 'overfitting_score', 'train_val_gap',
       'unstable_training', 'underfitting_flag', 'accuracy', 'precision',
       'recall', 'f1_score', 'roc_auc', 'failure_type', 'applied_augmentation',
       'rebalancing_method', 'architecture_change', 'threshold_optimization',
       'post_fix_f1_score'],
      dtype='object')
```

3. Remove Duplicates from DataFrame

This code initially uses df.duplicated().sum() to determine the number of duplicate rows in the DataFrame. Next, it uses df.drop_duplicates() to eliminate the duplicate rows. In order to verify that all duplicates have been eliminated from the dataset, the duplication count is finally examined once more.

Code:

```
duplicates = df.duplicated().sum()
df = df.drop_duplicates()
df.duplicated().sum()
```

output:

```
0
```

4.Code to Save and Load the Cleaned Dataset

This code uses df.to_csv() to save the cleaned dataset as a CSV file. The dataset is then saved in a new DataFrame named cleaned_df after being loaded once again using pd.read_csv(). Lastly, to verify that the dataset was correctly saved and loaded, cleaned_df.head() shows the first few rows.

Code:

```
df.to_csv("cleaned_ai_model_dataset.csv", index=False)

cleaned_df = pd.read_csv("cleaned_ai_model_dataset.csv")

print(cleaned_df.head())
```

output:

```
  project_id   year           domain   model_type  dataset_size  \
0      PRJ_1  0.875          finance      XGBoost      0.343732
1      PRJ_2  0.250  computer_vision          CNN      0.578184
2      PRJ_3  0.625          finance          CNN      0.080847
3      PRJ_4  0.750          finance     DenseNet      0.021200
4      PRJ_5  0.000          medical  Transformer      0.820372

   class_imbalance_ratio  label_noise_level  missing_data_ratio  data_leakage  \
0               0.362073           0.155859            0.155883           0.0
1               0.072665           0.456070            0.785180           0.0
2               0.388971           0.258675            0.662503           1.0
3               0.068276           0.280841            0.542655           0.0
4               0.215509           0.310905            0.325102           0.0

   overfitting_score  ...  precision    recall  f1_score   roc_auc  \
0           0.970004  ...   0.183288  0.304277  0.075352  0.524689
1           0.170212  ...   0.808525  0.304649  0.276552  0.097536
2           0.775128  ...   0.922047  0.088496  0.357343  0.195863
3           0.198418  ...   0.729104  0.771374  0.153924  0.073905
4           0.713209  ...   0.493800  0.522800  0.632287  0.427458

        failure_type  applied_augmentation  rebalancing_method  \
0   class_imbalance                   GAN                none
1  model_complexity                  none        oversampling
2  model_complexity                 SMOTE                none
3  model_complexity                   GAN                none
4       label_noise                   GAN        undersampling

   architecture_change  threshold_optimization  post_fix_f1_score
0                  0.0                     0.0           0.220989
1                  0.0                     0.0           0.393434
2                  0.0                     0.0           0.449160
3                  0.0                     0.0           0.120776
4                  0.0                     0.0           0.797671
```

2.2 TEXT DATA-PREPROCESSING:

This code cleans the dataset's category text columns.After choosing the columns domain, model_type, and failure_type, it uses.str.lower() and.str.strip() to transform all text values to lowercase and eliminate superfluous spaces. This guarantees that the data is kept in a uniform format.

Code:

```
text_cols = ['domain','model_type','failure_type']

for col in text_cols:

df[col] = df[col].str.lower().str.strip()

print(df[text_cols].head())
```

Output:

```
            domain   model_type      failure_type
0          finance      xgboost   class_imbalance
1  computer_vision          cnn  model_complexity
2          finance          cnn  model_complexity
3          finance     densenet  model_complexity
4          medical  transformer       label_noise
```

2.3 DATA AGGREGATION:

The average model performance for every domain is determined by this code.The dataset is grouped by the domain column using the groupby('domain') function. Next, it uses.mean() to get the mean (average) of the performance measures accuracy, precision, recall, and f_score for every domain.

Code:

```
domain_summary =
df.groupby('domain')[['accuracy','precision','recall','f_score']].mean()
print(domain_summary)
```

output:

```
                accuracy  precision    recall  f1_score
domain
computer_vision  0.700041   0.753873  0.753538  0.574842
finance          0.694778   0.742121  0.741540  0.568405
fraud            0.706046   0.747918  0.749978  0.581036
medical          0.701508   0.747196  0.746655  0.576893
nlp              0.701204   0.760439  0.755292  0.577160
recommender      0.704404   0.751843  0.758142  0.579590
```

2.4 NORMALIZATION AND SCALING

The dataset's numerical columns are normalized using this code. To scale values between 0 and , MinMaxScaler is first imported from the sklearn library. Next, the columns f_score, recall, accuracy, and precision are chosen and saved in num_cols. To normalize these columns, fit_transform() is used after creating the MinMaxScaler() object. Lastly, the top five rows of the scaled values are shown on the screen by print(df[num_cols].head()).

Code:

```
from sklearn.preprocessing import MinMaxScaler
num_cols = ['accuracy','precision','recall','f_score']
scaler = MinMaxScaler()
df[num_cols] = scaler.fit_transform(df[num_cols])
print(df[num_cols].head())
```

Output:

```
   accuracy  precision    recall  f1_score
0  0.058006   0.183288  0.304277  0.075352
1  0.199645   0.808525  0.304649  0.276552
2  0.311721   0.922047  0.088496  0.357343
3  0.140875   0.729104  0.771374  0.153924
4  0.729761   0.493800  0.522800  0.632287
```

2.5 DATA SPLITTING:

The dataset is divided into training and testing data by this code. The input features are stored in X, while the failure_type target column is stored in Y. The data is split into 80% training and 20% testing using the train_test_split() function. The size of the training and testing datasets is displayed using the print() commands.

Output:

```
X_train shape: (4800, 27)
X_test shape: (1200, 27)
y_train shape: (4800,)
y_test shape: (1200,)
```

3.DATA VISULIZATION

3.1 VISULIZATION:

Understanding patterns, relationships, and trends in the dataset is facilitated by data visualization, which is a crucial stage in data analysis. The performance of AI/ML models and the causes of model failures are examined in this study using various visualization techniques.It is simpler to evaluate the data and determine which elements have the greatest impact on model performance when graphical representations like bar charts, count plots, and heatmaps are used. Comparing model performance across many domains and failure kinds is another benefit of visualization.In this project, visuals are made using Python tools like Matplotlib and Seaborn. These packages offer strong tools for creating understandable and instructive graphs.

1.Most Used Model:

The most popular AI/ML model in the dataset is found and displayed by this code. Value_counts() first counts the number of times each model type appears; the most frequent model and its count are then shown using idxmax() and max(). Lastly, the usage of all models is represented in a bar chart. This figure offers insight into model popularity and focus areas in the dataset, making it easier to immediately determine which model types are most frequently employed in certain trials.

Code:

```
most_used_model = df['model_type'].value_counts()
print("Most used model:")
print(most_used_model.idxmax(), "→", most_used_model.max())
most_used_model.plot(kind='bar', title="Model Usage Count")
plt.show()
```

Output:

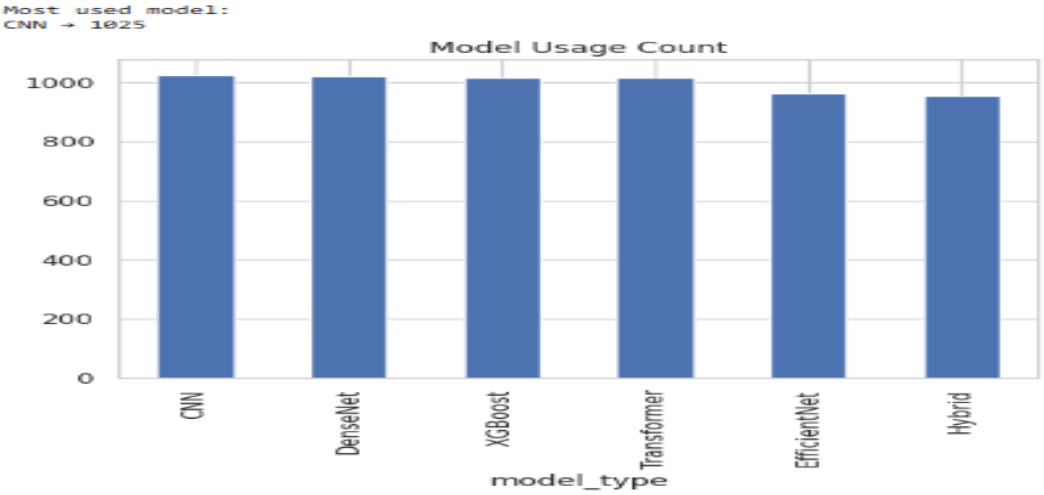

Fig.1. Most Used Model

2.AI Model Type vs Failure Count:

The amount of problems for each type of AI/ML model is displayed in a bar chart created by this code. The unique model values in the dataset are first shown by df['model_type'].unique(). Next, numerical values are translated into model names like CNN, RNN, Random Forest, SVM, and Logistic Regression using a dictionary named model_map. These model names are substituted for the numbers by the map() function. Value_counts() then determines how frequently each model type occurs in the dataset. Lastly, a bar chart displaying the quantity of problems for each model type is made using plot(kind='bar'). This image facilitates the comparison of model performance and helps determine which AI models encounter more failures.

Code:

```
print(df['model_type'].unique())
model_map = {
    0: "CNN",
    : "RNN",
    2: "Random Forest",
    3: "SVM",
    4: "Logistic Regression"
}
df['model_type'] = df['model_type'].map(model_map)
model_issues = df['model_type'].value_counts()
```

```
model_issues.plot(kind='bar')
plt.title("AI Model Type vs Failure Count")
plt.xlabel("Model Type")
plt.ylabel("Number of Issues")
plt.xticks(rotation=45)
plt.show()
```

Output:

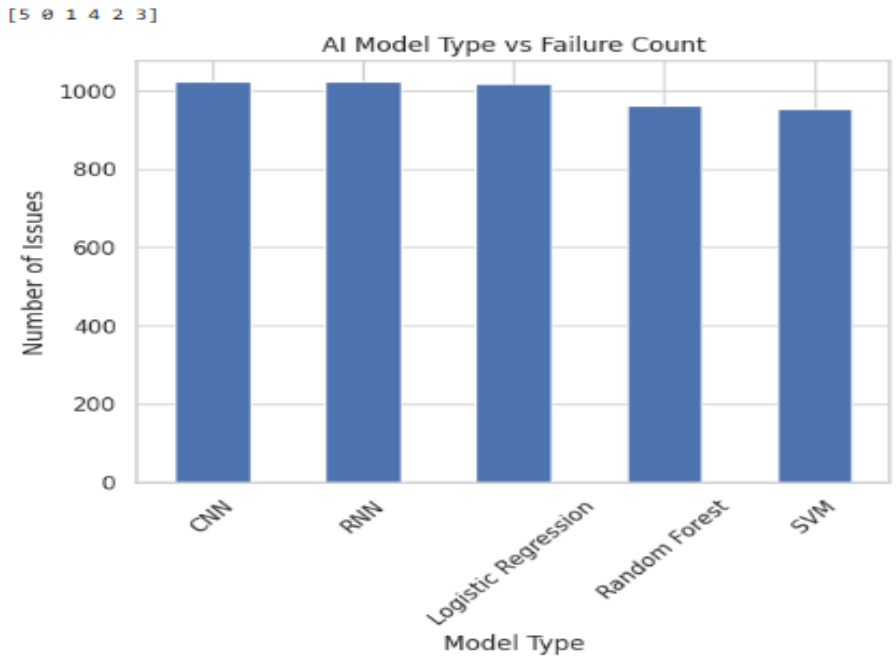

Fig.2.AI Model Type vs Failure Count

3. Failure Type Distribution:

The code provided uses a pie chart to show the prevalence of AI model failure categories. The failure_type column's unique values are first examined. To translate numerical codes to descriptive failure descriptors like Overfitting, Underfitting, Data Leakage, High Noise, and Class Imbalance, a dictionary failure_map is developed. These labels are substituted for the numerical codes by the map() function, and value_counts() counts the frequency of each failure category. Lastly, a pie chart with percentage labels and the caption "AI Model Failure Types" is created using Matplotlib to display the percentage of each failure category in the dataset. The most frequent errors in the dataset can be clearly understood thanks to this depiction.

Code:

```
print(df['failure_type'].unique())
failure_map = {
    0: "Overfitting",
    1: "Underfitting",
    2: "Data Leakage",
    3: "High Noise",
    4: "Class Imbalance"
}
df['failure_type'] = df['failure_type'].map(failure_map)
failure_counts = df['failure_type'].value_counts()
plt.figure(figsize=(6,6))
plt.pie(failure_counts,
labels=failure_counts.index,
autopct='%.f%%',
startangle=40)
plt.title("AI Model Failure Type Distribution")
plt.show()
```

Output:

```
[0 4 3 2 5 1]
```

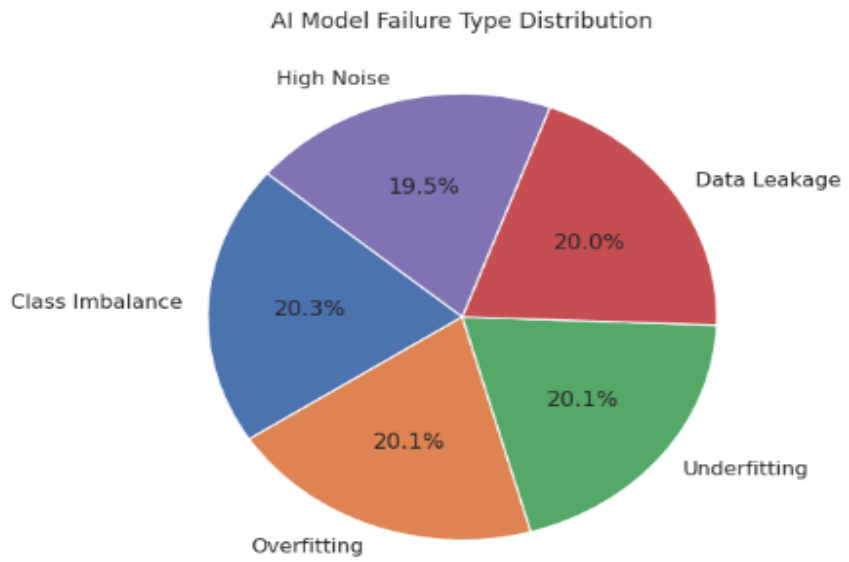

Fig. 3. Failure Type Distribution

4. Domain Distribution Visualization:

The distribution of AI/ML models across various fields is displayed in a bar chart produced by this code. The number of models in each domain is counted via the Seaborn library's countplot() function. The chart size is set by plt.figure(figsize=(8,5)), the x-axis labels are rotated for readability by plt.xticks(rotation=45), and a descriptive title is added by plt.title(). Each bar is shown in a different color when the palette parameter is added, improving the chart's visual attractiveness.With identifying the domains with the greatest number of AI/ML trials, this visualization sheds light on the fields in which machine learning models are most frequently used. It is the initial stage of comprehending the domain distribution and dataset structure.

Code:

```
plt.figure(figsize=(8,5))

sns.countplot(x='domain', data=df, palette='Set2')

plt.title("Distribution of Models Across Domains")

plt.xticks(rotation=45)

plt.show()
```

Output:

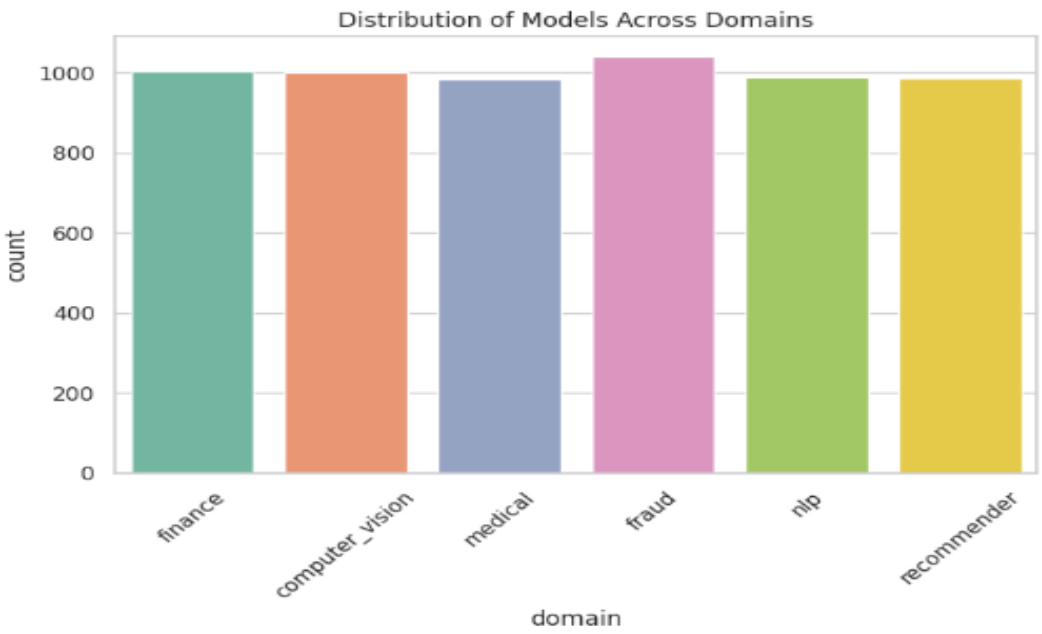

Fig.4. Domain Distribution Visualization

5. Recovery Techniques Usage:

This code generates a bar chart that displays the dataset's use of various recovery strategies. The dataset is first loaded into a DataFrame, and for easier reading, the rebalancing_method column is transferred to more descriptive names like "No Rebalancing" and "Random Oversampling." The code then creates a colored bar chart based on the number of times each recovery approach is utilized.This graphic is used to illustrate which recovery techniques are most frequently used to enhance the performance of AI/ML models. This makes it easier to comprehend how frequently each method is applied and offers information about the recovery tactics that work best in certain studies.

Code:

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv("/content/synthetic_ai_model_failure_recovery.csv")
display_name_map = {
    'none': 'No Rebalancing',
    'oversampling': 'Random Oversampling',
    'undersampling': 'Random Undersampling',
    'class_weight': 'Class Weighting',
}
solution_usage = df['rebalancing_method'].value_counts()
solution_usage = solution_usage.rename(index=display_name_map)
colors = ['#FF6F6', '#6B5B95', '#88B04B', '#FFA500', '#00CED']
solution_usage.plot(kind='bar', color=colors, edgecolor='black')
plt.xlabel("Recovery Method")
plt.ylabel("Count")
plt.title("Usage of Recovery Techniques")
plt.xticks(rotation=45)
plt.show()
```

Output:

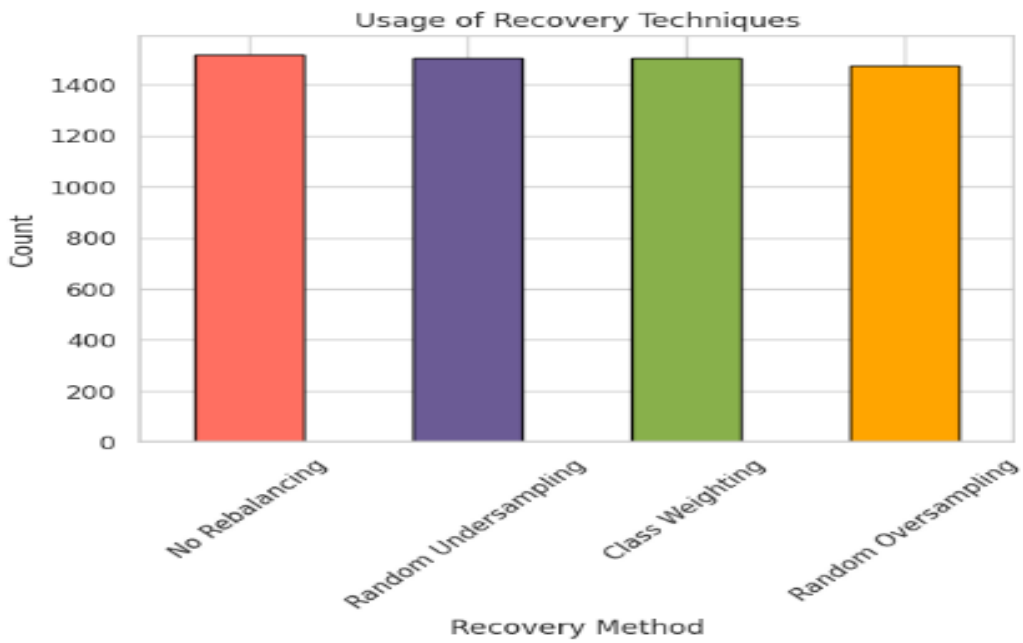

Fig.5 Recovery Techniques Usage

6. Performance Before vs After Solution:

This graph illustrates how AI/ML models' performance has improved over time. To make sure the timeline is accurate, the dataset is first sorted by year. The F-score prior to the use of recovery techniques and the F-score following the application of recovery strategies are represented by two lines. This makes it easier to see how utilizing recovery techniques enhances the models' performance. The graph illustrates the efficacy of recovery strategies in boosting model reliability and is helpful in understanding patterns in model improvement over time.

Code:

```
df_sorted = df.sort_values('year')
plt.figure(figsize=(7,4))
plt.plot(df_sorted['year'], df_sorted['f_score'], label='Before Fix')
plt.plot(df_sorted['year'], df_sorted['post_fix_f_score'], label='After Fix')
plt.xlabel("Year")
plt.ylabel("F-score")
plt.title("Performance Improvement Over Time")
plt.legend()
plt.show()
```

Output:

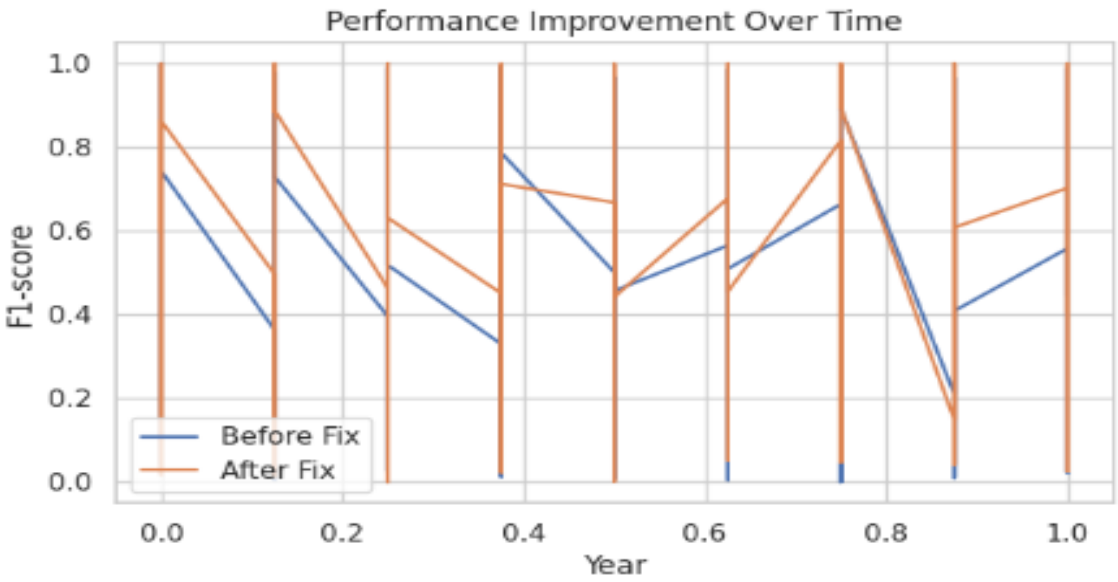

Fig.6.Performance Before vs After Solution

7. Overall Problem vs Solved Comparison:

In order to examine the average F-score of AI/ML models before and after using recovery approaches, this code generates a bar chart. For clarity, it depicts the mean values of the f_score and post_fix_f_score columns using various colors. The graphic makes it simple to visually illustrate the efficacy of recovery techniques in the project analysis by showing how model performance improves following recovery.

Code:

```
import matplotlib.pyplot as plt
comparison = df[['f_score','post_fix_f_score']].mean()
comparison.plot(
    kind='bar',
    figsize=(5,4),
    color=['skyblue', 'orange']   # Different colors here
)
plt.ylabel("Average F-score")
plt.title("Model Performance: Before vs After Recovery")
plt.xticks(rotation=0)
```

```
plt.show()
```

Output:

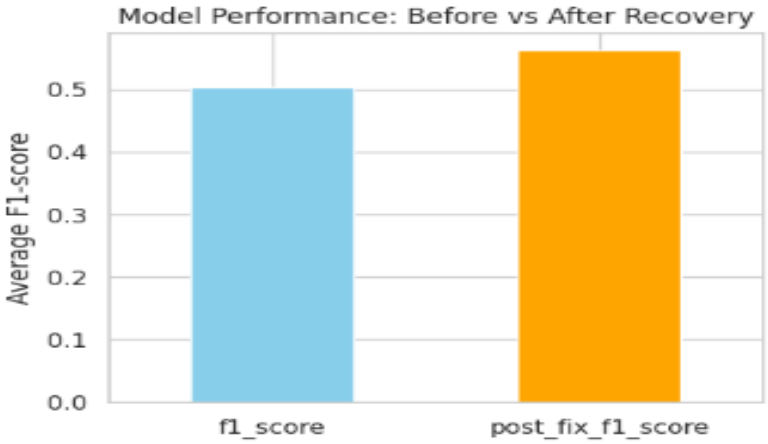

Fig.7.Overall Problem vs Solved Comparison

3.2 CORRELATION

The correlation heatmap illustrates the relationships between the dataset's numerical features. It aids in determining correlations between other numerical columns like class imbalance ratio, label noise, and overfitting score and performance measures like accuracy, precision, recall, and F1-score in this project. It is easier to concentrate on the areas that require improvement when we can see which variables are most closely linked to model failures and performance.

1.Performance Metrics Correlation:

The correlations between various performance indicators, including accuracy, precision, recall, F1-score, ROC-AUC, and post-fix F1-score, are displayed in this heatmap. It assists in determining the degree of correlation between certain metrics, such as if enhancing one statistic also enhances others. We may have a better understanding of the general performance trends of AI/ML models and how recovery strategies impact several parameters at once by using this figure.

Code:

```
perf_cols = ['accuracy','precision','recall',
             'f1_score','roc_auc','post_fix_f1_score']
sns.heatmap(df[perf_cols].corr(), annot=True, cmap='viridis')
plt.title("Correlation Between Performance Metrics")
plt.show()
```

Output:

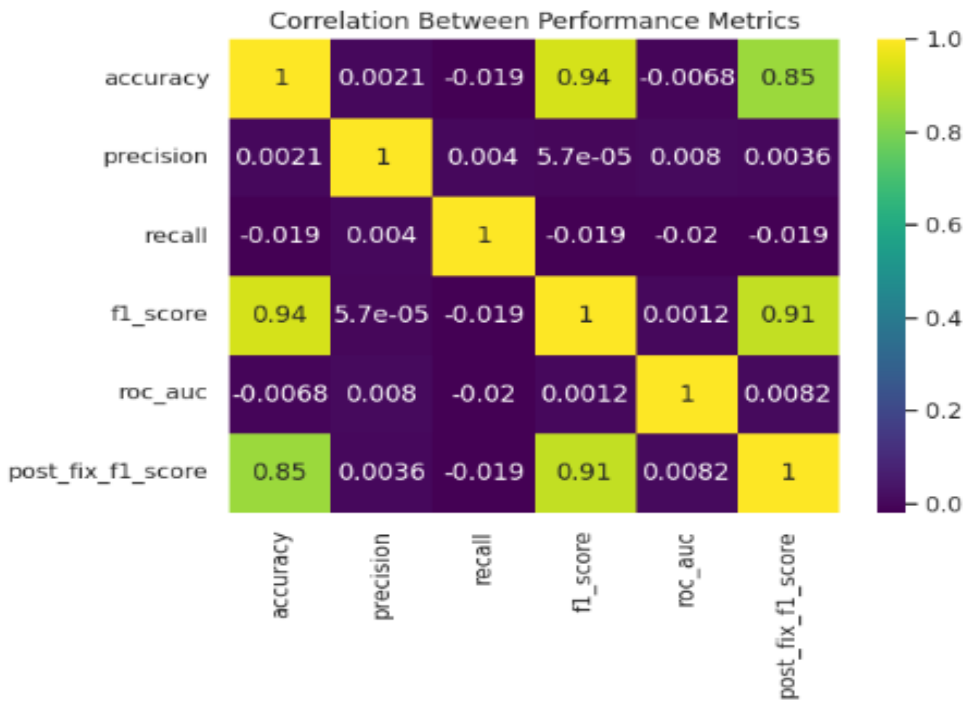

Fig.8.Performance Metrics Correlation

4.1 CONCLUSION

The Synthetic AI Model Failure Recovery Dataset was examined in this project, Analysis of AI/ML Model Failures and Performance Improvement Using Recovery Techniques, in order to determine the common causes of AI and machine learning model failures and to assess how recovery techniques can enhance model performance.

According to the analysis, deep learning models like CNN and Transformer had the highest failure rate when compared to other models like XGBoost. This is mostly because these models need a lot of clean, balanced training data. Because healthcare datasets frequently contain label noise, missing values, and class imbalance, which impact the models' learning process, the medical domain in particular saw a higher number of model failures. Similar to how class imbalance and data leakage caused failures in the finance sector, overfitting and unstable training caused models like CNN to routinely fail in the computer vision domain when the dataset was insufficiently large or diverse. A number of recovery strategies were used to address these issues, such as SMOTE and oversampling approaches to address class imbalance, data augmentation to boost training data, architecture modifications to stabilize deep learning models, and threshold adjustment to increase prediction accuracy.

Evaluation criteria like accuracy, precision, recall, and F1-score were used to quantify how much the models' performance improved after different recovery strategies were applied. Applying suitable recovery approaches can successfully reduce model failures and increase the reliability of AI/ML systems across various domains, as demonstrated by the comparison between the original model performance and the post-fix F1-score.

4.2 FUTURE ENHANCEMENT

The Synthetic AI Model Failure Recovery Dataset analysis in the project Analysis of AI/ML Model Failures and Performance Improvement Using Recovery Techniques assisted in determining the primary reasons for AI/ML model failures as well as the recovery strategies employed to enhance their performance. To further increase the system's accuracy, dependability, and practicality, a number of improvements can be done in the future.

1.Utilizing Real-World Datasets: In this study, an artificial dataset was analyzed. Real-world datasets from fields like computer vision, healthcare, and finance may be utilized in the future. This will enhance the dependability of the outcomes and aid in comprehending actual AI model failures.

2.Advanced Machine Learning Model Implementation: It is possible to construct and compare more sophisticated AI models and systems. It is possible to determine which deep learning models are more reliable and less likely to fail in certain areas by testing more models.

3.Systems for Automated Failure Detection: Future research could concentrate on developing automated systems that can track model performance and instantly identify malfunctions. These systems are capable of automatically detecting problems including unstable training, data imbalance, and overfitting.

4.Better Recovery Methods: To further enhance model performance and lower failure rates, advanced recovery strategies such enhanced data augmentation, hyperparameter tuning, and ensemble learning techniques can be used.

5.Evaluation of Cross-Domain Performance: Future studies can examine AI model failures in more detail across other domains. This will assist in identifying problems unique to each domain and creating recovery plans tailored to each one.

4.3 REFERENCE

[1] Kaggle – Synthetic AI Model Failure Recovery Dataset, https://www.kaggle.com/datasets

[2] Kaggle – Kaggle Machine Learning and Data Science Platform, https://www.kaggle.com/

[3] Kaggle – Kaggle Notebooks for Data Analysis and Machine Learning Experiments ,https://www.kaggle.com/code

[4] Kaggle – Kaggle Datasets Repository ,https://www.kaggle.com/datasets

[5] Kaggle – Kaggle Machine Learning Tutorials and Community Resources, https://www.kaggle.com/learn

[6] Scikit-learn Documentation – Machine Learning Algorithms and Model Evaluation, https://scikit-learn.org/stable/

[7] TensorFlow Documentation – Machine Learning Training and Optimization, https://www.tensorflow.org/

[8] Pandas Documentation – Data Analysis and Data Manipulation Tools, https://pandas.pydata.org/docs/